Programación Avanzada
IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha

Anuncios

09 de abril de 2026

1. Hoy tendremos la Actividad 4.
2. Recuerden que la Tarea 1 se entrega mañana (viernes 10 de abril a las 20.00 hrs).
3. Recuerden que la ECA abre de domingo a martes. Si responden 10 veces tienen un bonus en el promedio final.

Levantamiento de Excepciones

Levantamiento de Excepciones

- ¿Qué son las Excepciones?
- ¿Cómo se levanta una Excepción?
- ¿Qué pasa cuando se levanta una Excepción?

Ejemplo Traceback

main.py

```python
def func1():
    return func2()

def func2():
    return func3()

def func3():
    result = 10 / 0
    return result

if __name__ == '__main__':
    func1()

```

Terminal

```
> python main.py
Traceback (most recent call last):
  File "main.py", line 13, in <module>
    func1()
File "main.py", line 2, in func1
    return func2()
           ^^^^^^^
  File "main.py", line 5, in func2
    return func3()
           ^^^^^^^
  File "main.py", line 8, in func3
    result = 10 / 0
             ~~~^~~
ZeroDivisionError: division by zero
```

Ejemplo raise

main.py

```python
def dividir(a: int, b: int) -> int:
    if b == 0:
        raise ValueError("No se puede dividir por cero")
    return a / b

numeros = [(10, 2), (5, 0), (8, 4)]
for x, y in numeros:
    try:
        r = dividir(x, y)
        print(f"{x} ÷ {y} = {r}")
    except ValueError as e:
        print(f"⚠️ {e}")

```

Terminal

```
> python main.py
10 ÷ 2 = 5.0
⚠️ No se puede dividir por cero
8 ÷ 4 = 2.0
```

Manejo de Excepciones

Manejo de Excepciones

- ¿Cómo se maneja una Excepción?
- En Python, ¿es posible manejar todas las Excepciones?
- ¿Cuántas Excepciones puede atrapar un bloque `try/except`?
- ¿Cómo complementan las sentencias `else` y `finally` un bloque `try/except`?
- ¿Es buena práctica hacer "`except Exception:`" o solo "`except:`"?
- ¿Es mejor usar `if/else` o `try/except`?

Iterables e Iteradores

Tip: Es más fácil acordarse si uno piensa que son recorribles y recorredores, respectivamente.

Iterables e iteradores: Aplicando en código

¿Cuál es el iterable y cuál el iterador? ¿Cuál funciona de la forma deseada aquí?

```
dias = ["no sube", "no sube", "sube mañana", "sube", "ya subió"]
texto = "Revisando $ bencina:"
```

```
copy_dias = dias.copy()

for dia in copy_dias:
    if dia == "sube mañana":
        break

for dia in copy_dias:
    print(texto, dia)
```

```
iter_dias = iter(dias)

for dia in iter_dias:
    if dia == "sube mañana":
        break

for dia in iter_dias:
    print(texto, dia)
```

Iterables e iteradores: Aplicando en código

¿Cuál es el iterable y cuál el iterador? ¿Cuál funciona de la forma deseada aquí?

```
dias = ["no sube", "no sube", "sube mañana", "sube", "ya subió"]
texto = "Revisando $ bencina:"
```

```
copy_dias = dias.copy()

for dia in copy_dias:
    if dia == "sube mañana":
        break

for dia in copy_dias:
    print(texto, dia)
```

```
Revisando $ bencina: no sube
Revisando $ bencina: no sube
Revisando $ bencina: sube mañana
Revisando $ bencina: sube
Revisando $ bencina: ya subió
```

Iterables e iteradores: Aplicando en código

¿Cuál es el iterable y cuál el iterador? ¿Cuál funciona de la forma deseada aquí?

```
dias = ["no sube", "no sube", "sube mañana", "sube", "ya subió"]
texto = "Revisando $ bencina:"
```

```
Revisando $ bencina: sube
Revisando $ bencina: ya subió
```

```
iter_dias = iter(dias)

for dia in iter_dias:
    if dia == "sube mañana":
        break

for dia in iter_dias:
    print(texto, dia)
```

¿Iterador = Iterable?

NO, pero en Python un Iterador puede funcionar como Iterable.
Para entender mejor esto, veamos algunas de sus principales diferencias:

Característica	Iterable	Iterador
Métodos Asociados	Implementa el método **`__iter__()`** que devuelve un iterador, o el método **`__getitem__()`** que va entregando elementos de forma creciente.	Implementa el método **`__next__()`**. Es el objeto retornado de un **`__iter__()`**.
Estado Interno	Un iterable **no tiene estado interno** de iteración.	Un iterador **tiene un estado interno** que recuerda su posición actual en la iteración.
Reutilización	Los iterables **pueden ser reutilizados** para obtener múltiples iteradores.	Los iteradores, una vez consumidos, **no pueden ser reutilizados**.

Generadores

Generadores

- ¿Cuál es la ventaja de usar generadores?

Generadores

```
def cuadrado(numero):
    print("Procesando", numero)
    return numero**2

lista = [cuadrado(num) for num in range(3)]
gen = (cuadrado(num) for num in range(3))

for num in gen:
    print("Gen", num)
```

```
Procesando 0
Procesando 1
Procesando 2
```

Generadores

```
def cuadrado(numero):
    print("Procesando", n)
    return n**2

lista = [cuadrado(num) for num in range(3)]
gen = (cuadrado(num) for num in range(3))

for num in gen:
    print("Gen", num)
```

Generadores

```
def cuadrado(numero):
    print("Procesando", n)
    return n**2

lista = [cuadrado(num) for num in range(3)]
gen = (cuadrado(num) for num in range(3))

for num in gen:
    print("Gen", num)
```

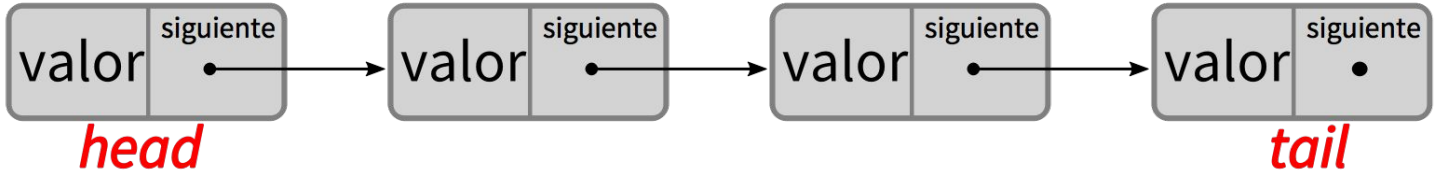
```
Procesando 0
Gen 0
Procesando 1
Gen 1
Procesando 2
Gen 4
```

Listas Ligadas

Listas Ligadas

- ¿Para qué sirve este tipo de estructura?
- ¿En qué se diferencia de una lista de Python?

👉 Para más información revisar: [4-listas-ligadas.ipynb]



Pregunta Evaluación Escrita

Veamos una pregunta de Evaluación Escrita

(Examen 2024-1)

15. Considere el siguiente código en python. Sin considerar saltos de línea, ¿qué imprime este código?

```python
try:
    resultado = 10 / 0
    print("TRY")
except ZeroDivisionError:
    print("EXCEPT")
else:
    print("ELSE")
finally:
    print("FINALLY")
```

A) EXCEPT FINALLY
B) ELSE FINALLY
C) EXCEPT FINALLY TRY
D) TRY EXCEPT FINALLY
E) TRY ELSE FINALLY

Veamos una pregunta de Evaluación Escrita

(Examen 2024-1)

15. Considere el siguiente código en python. Sin considerar saltos de línea, ¿qué imprime este código?

```python
try:
    resultado = 10 / 0
    print("TRY")
except ZeroDivisionError:
    print("EXCEPT")
else:
    print("ELSE")
finally:
    print("FINALLY")
```

A)	**EXCEPT**	**FINALLY**	
B)	ELSE	FINALLY	
C)	EXCEPT	FINALLY	TRY
D)	TRY	EXCEPT	FINALLY
E)	TRY	ELSE	FINALLY

Pregunta de Evaluación Escrita

Tema: Iterables e iteradores (Midterm 2025-1)

5. ¿Cuál de los siguientes tipos de datos **no** presenta un iterador?

   A) *Strings*
   B) Conjuntos
   C) Tuplas
   D) Diccionarios
   E) Todos los tipos de datos anteriores presentan un iterador.

Pregunta de Evaluación Escrita

Tema: Iterables e iteradores (Midterm 2025-1)

5. ¿Cuál de los siguientes tipos de datos **no** presenta un iterador?

   A) *Strings*
   B) Conjuntos
   C) Tuplas
   D) Diccionarios
   E) Todos los tipos de datos anteriores presentan un iterador.

Actividad

Comentarios AC4

Hagan `git add`, `git commit`, `git push`

Programación Avanzada
IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha